

I built an AI-SDLC framework for a regulated industry.

Then I ran it on a real repo — and it failed its own core test.

THE PROBLEM

Once agents write most of the diff, two things break.

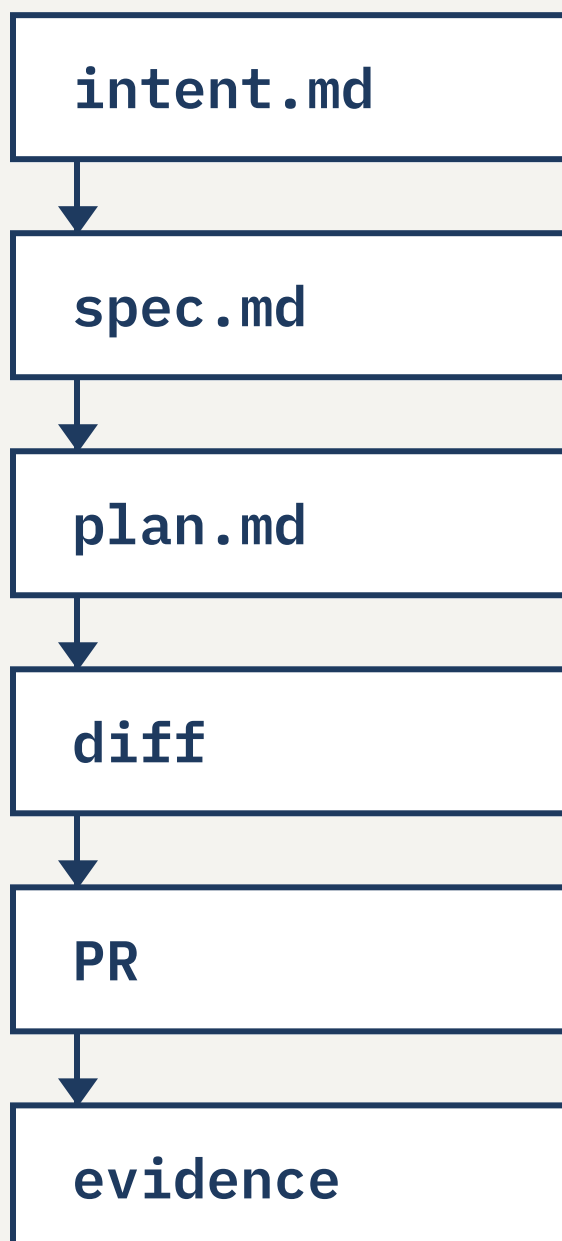
Review capacity, and evidence.

Going faster doesn't remove the bottleneck. It just moves the queue to QA.

And the human approval gate that's supposed to catch what the agent got wrong quietly becomes fiction — someone signs off on a volume of change nobody actually read.

THE INSIGHT

The chain of commits is the audit trail.



Each stage commits an artifact the next stage has to read. Evidence gets derived from what actually happened — not assembled afterward, at release.

No skill gets to assume a stack fact.

Every claim about the stack has to come from the code — grepped, evidenced, and marked **[confirmed]** / **[inferred]** / **[ASK]**.

Not from a README. Not from CLAUDE.md.

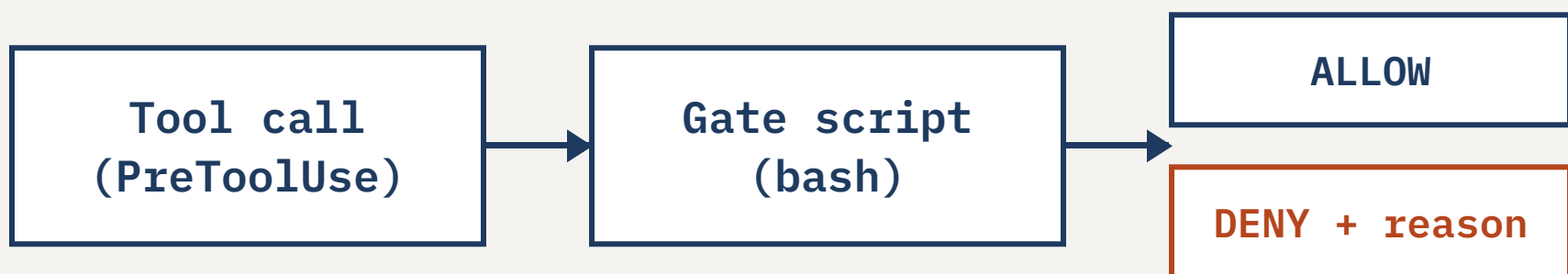
An unresolved [ASK] on the change's own dependency path blocks the change until a human answers it.

SKILLS ADVISE. HOOKS ENFORCE.

A skill can suggest. Only a hook can block.

Skills read context and recommend — advisory, not binding.

Policy lives in deterministic scripts that run on every tool call and can deny it outright. That split is what makes enforcement real instead of best-effort.



WHERE IT WORKED

It asked the repo. Not the README.

CLAUDE.md said the stack included SQLAlchemy, Alembic, PyJWT, passlib.

Discovery grepped the actual code: zero imports of any of them, anywhere.

The README got corrected. Not the other way around.

SQLAlchemy

Alembic

PyJWT

passlib

The zip that ships the plugins strips the execute bit.

Claude Code treats a hook execution failure as **allow**.

Every gate script that lost its +x on extraction stopped enforcing — silently, on install, for every adopter.

A gate that fails without telling you is worse than no gate. People keep trusting it.

The evidence tool had the evidence bug.

The CLI I built to prove requirements have real test coverage reported clean on this exact repo.

It trusted a spreadsheet row claiming a test existed — without checking the test file actually said so.

```
before: 0 uncovered · exit 0  
after:  3 uncovered · exit 1
```

Reporting clean because it never looked — exactly what the framework exists to catch.

WHERE THIS ACTUALLY SITS

This is not the well-funded option.

Ketryx has raised \$57M and is trusted by 3 of the top 5 MedTech companies.

At least five other open-source frameworks cover similar ground.

The honest claim here is narrow: the vendor-neutral, self-hosted option for teams who can't or won't buy a platform.

WHAT I'D TELL SOMEONE STARTING THIS

Run it on your worst repo. Not your cleanest one.

Test the execute bit on install, not after an incident report.

Hold your own tooling to the same evidence bar you're asking of everyone else's code.

And run it somewhere with real gaps already. A framework that never finds a problem hasn't been tested — it's been demoed.

ONE QUESTION

If you're running validated software with AI in the loop — when did you last check whether your own evidence tooling has the exact bug it's supposed to catch?